

TALLINN UNIVERSITY OF TECHNOLOGY
IT College

Marko Linde 176292IADB

CALCULATOR

Javascript (ICD0006)

Instructor: Andres Käver

Tallinn 2022

Table of Contents

1 Assignment	3
2 Overview	3
2.1 Calculator logic.....	3
2.2 Display.....	4
2.3 Functionality.....	5
2.3.1 Main.....	5
2.3.2 Evaluations after calculate command.....	5
2.3.3 Calculation command.....	5
Annex 1 – index.js	6

1 Assignment

Write simple calculator using Bootstrap (or any other css lib) based design and simple js functionality.

Use only js functionality learned in class.

Provide basic functions - +, -, *, /, C

2 Overview

Index.html and index.css taken from calculator example . Index.js is written by document author.

2.1 Calculator logic

Basic idea comes from logic, that the functionality of the calculator can be expanded.

- All calculator clicks are saved in a long string only array which allows:
 - to display the entire equation to the user

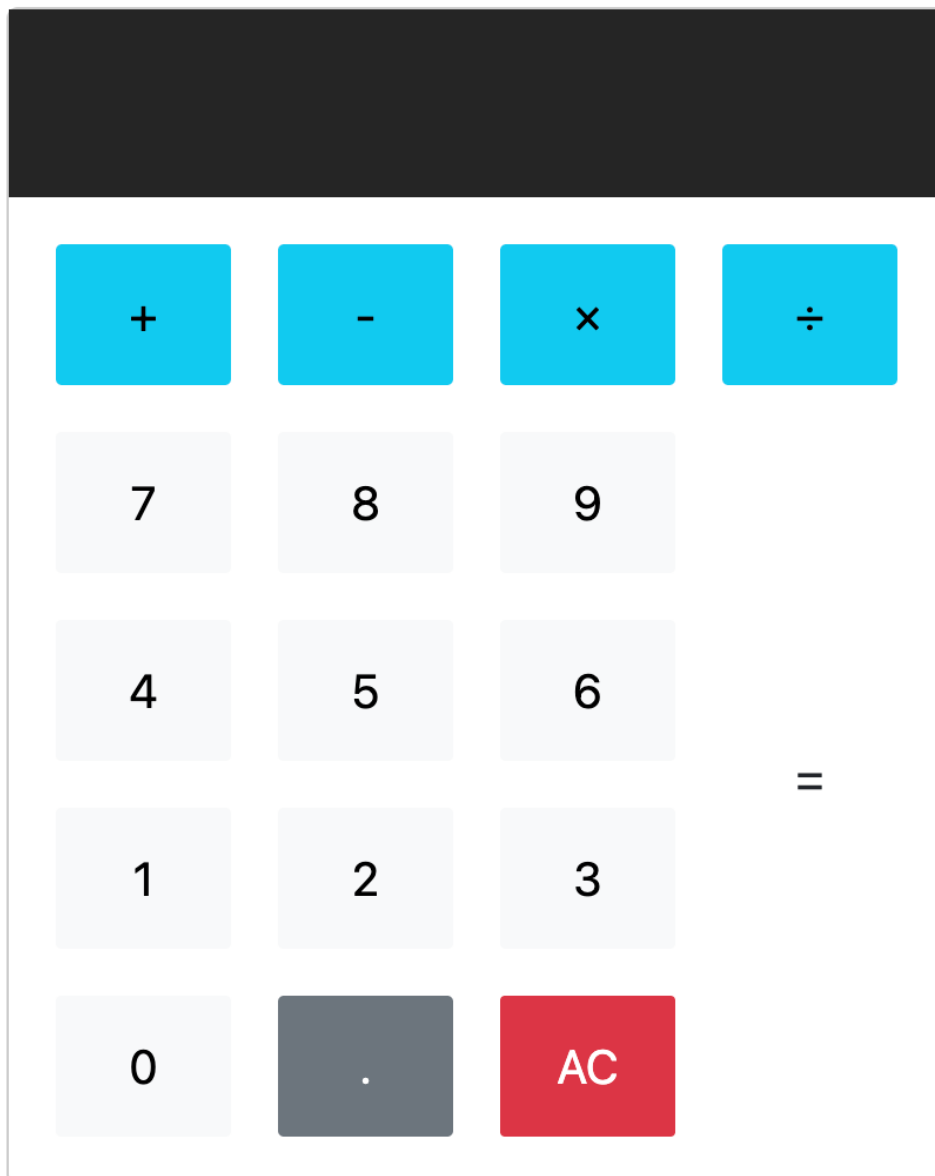


Drawing 1. Equation view

- to add undo button functionality
- to add calculate only parts of the equation functionality
- to add click on equation element and repair it directly functionality
- to make calculator more scientific

- Calculator starts to calculate only when clicking the equal sign
- Calculations are done from left to right with rule: first multiply or divide then add or subtract

2.2 Display



Drawing 2. Calculator screen

2.3 Functionality

2.3.1 Main

- All buttons have class “btn” and are observed through it.
- All buttons have distinct values assigned to them and differentiated by those values
- Every button click value is inserted into equations array separately (needs to change at next iteration for evaluate right away functionality - is faster and simpler)
- Equal button clicking calls calculation function
- AC button click calls overwriting of the previous array by empty array

2.3.2 Evaluations after calculate command

- Minus sign at the start is converted to a minus and number at the start and since the numbers are strings, even -0 can be saved correctly.
- Operators the start and end equation are evaluated – that they don’t exist

2.3.3 Calculation command

- Calculation operator is searched from the equation and its index is returned.
- The previous and after numbers are generated and the subset calculation is performed
- Calculated value is inserted back into the equation
- New calculation command iteration will begin

Annex 1 – index.js

```
"use strict";
/**/import css from "./index.css";

/*https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelectorAll */
/*https://sebhastian.com/javascript-queryselectorall/ */
const buttons = document.querySelectorAll('.btn');
const screen = document.querySelector('.calculator-screen');
const operators = ["+", "-", "/", "*"];

let equation = [];

buttons.forEach(button =>{
  button.onclick = function(event){
    let btnValue = button.getAttribute('value');
    console.log(btnValue);

    switch(btnValue) {
      case 'all-clear':
        clearEquationElement();
        break;
      case '=':
        convertFirstMinusToFullNegative();
        calculateValue(equation);
        break;
      default:
        addEquationElement(btnValue);
    }
    displayOnScreen(concatParams(equation));
  }
})
```

```

function calculateValue(equation){
    while(countSymbolsInEquation(operators)>0){
        calculateIndexValue(getNextCalculationIndex());
    }
}

function convertFirstMinusToFullNegative(){
    if ((equation[0] === '-') && (Number.isInteger(parseInt(equation[1])))){
        equation.splice(0,2,equation[0].concat(equation[1]));}
    else {
        clearEquationElement();
        displayOnScreen(equation);
    }
    console.log('Equation: ' + JSON.stringify(equation));
}

function countSymbolsInEquation(operators){
    let symbolCount = 0;
    let counter = 0;
    let symbolError = 0;
    equation.forEach(elem => {
        counter +=1;
        operators.forEach(operator =>{
            if (elem === operator) symbolCount+=1;
            if (((elem === operator) && (counter === 1)) ||
                ((elem === operator) && (counter === equation.length))) {symbolError = 1};
        })
    })
    console.log('Symbol error: ' + symbolError);
    console.log('Symbols present: ' + symbolCount);
    return (symbolError ? 0 : symbolCount);
}

```

```

function getNextCalculationIndex(){
    let startIndex = NaN;

    if (((getIndex('*') !== -1) && (getIndex('*') < getIndex('/'))) ||
        ((getIndex('/') === -1) && (getIndex('*') > getIndex('/')))) startIndex = getIndex('*');
    else if (getIndex('/') >= 1) startIndex = getIndex('/');
    else if (getIndex('+') >= 1) startIndex = getIndex('+');
    else if (getIndex('-') >= 1) startIndex = getIndex('-');

    console.log('Starting with index: ' + startIndex);
    return startIndex;
}

function calculateIndexValue(startIndex){
    let value = NaN;

    switch(equation[startIndex]) {
        case '*':
            value = parseFloat(getPreviousNumber(startIndex, operators)) * parseFloat(getNextNumber(startIndex, operators));
            break;
        case '/':
            value = parseFloat(getPreviousNumber(startIndex, operators)) / parseFloat(getNextNumber(startIndex, operators));
            break;
        case '+':
            value = parseFloat(getPreviousNumber(startIndex, operators)) + parseFloat(getNextNumber(startIndex, operators));
            break;
        case '-':
            value = parseFloat(getPreviousNumber(startIndex, operators)) - parseFloat(getNextNumber(startIndex, operators));
            break;
        default:
            value;
    }

    console.log('Index calculation value: ' + value);
    insertCalculationToEquation(startIndex, operators, value.toString());
    console.log('Equation: ' + JSON.stringify(equation));
}

```



```

function insertCalculationToEquation(startIndex, operators, value){
    let previousIndex = getPreviousSymbolIndex(startIndex, operators);
    let nextIndex = getNextSymbolIndex(startIndex, operators);

    console.log('previousIndex: ' + previousIndex);
    console.log('nextIndex: ' + nextIndex);

    if (startIndex >= 0) equation.splice(previousIndex + 1,nextIndex-previousIndex-1,value);
}

function getPreviousSymbolIndex(startIndex, findOperators){
    let previousIndex = startIndex-1;
    while(!(findOperators.includes(equation[previousIndex])) && previousIndex >= 0){
        previousIndex--;
    }
    return previousIndex;
}

function getNextSymbolIndex(startIndex, findOperators){
    let nextIndex = startIndex+1;
    while(!(findOperators.includes(equation[nextIndex])) && nextIndex < equation.length){
        nextIndex++;
    }
    return nextIndex;
}

function getPreviousNumber(startIndex, findOperators){
    let previousIndex = startIndex-1;
    let previousNumber = NaN;
    while(!(findOperators.includes(equation[previousIndex])) && previousIndex >= 0){

        if (!isNaN(previousNumber)) {previousNumber = equation[previousIndex].concat(previousNumber);}
        else {previousNumber = equation[previousIndex];}
        previousIndex--;
    }
    return previousNumber;
}

```

```

function getNextNumber(startIndex, findOperators){
    let nextIndex = startIndex+1;
    let nextNumber = NaN;

    while(!(findOperators.includes(equation[nextIndex])) && nextIndex < equation.length){

        if (!isNaN(nextNumber)) {nextNumber = nextNumber.concat(equation[nextIndex]);}
        else {nextNumber= equation[nextIndex];}
        nextIndex++;
    }
    return nextNumber;
}

function getIndex(sign){
    return equation.findIndex((element) => element === sign);
}

function addEquationElement(elem){
    equation.push(elem);
    console.log(JSON.stringify(equation));
}

function clearEquationElement(){
    equation = [];
    console.log(JSON.stringify(equation));
}

function displayOnScreen(value){
    screen.value = value;
}

function concatParams(params){
    return params.join('');
}

```